

JavaScript

Cơ bản về JavaScript

Javascript là gì?

- JavaScript là một ngôn ngữ lập trình script (kịch bản).
- Thực thi trên các trình duyệt (Chrome, Firefox,...) thông qua các Javascript Engine.
- Là ngôn ngữ thuộc loại loosely-typed.
- Sử dụng với mục đích chính là để tương tác với các thẻ HTML (DOM elements).

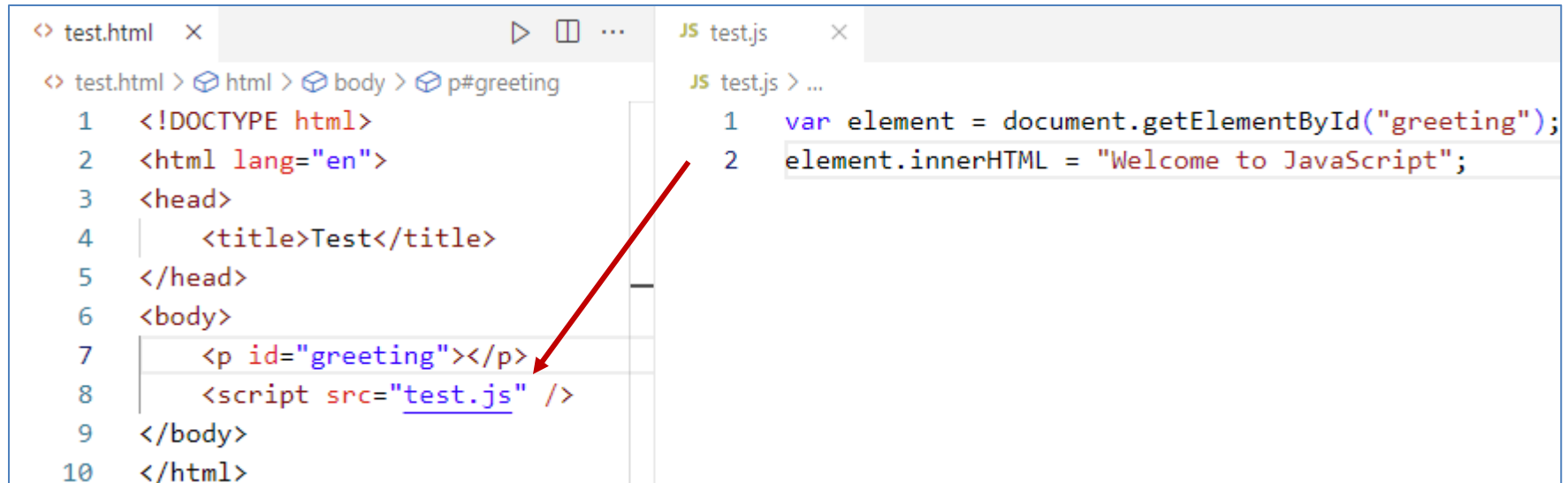
Cách viết mã lệnh JavaScript

- Mã lệnh được viết trong cặp thẻ `<script>` `</script>` và được chèn trong file HTML (Internal JavaScript).
 - Thẻ `<script>` có thể chèn bất cứ vị trí nào trong file HTML.
 - Có thể chèn nhiều thẻ `<script>` trong file HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Test</title>
</head>
<body>
  <p id="greeting"></p>
  <script>
    document.getElementById("greeting").innerHTML = "Welcome to JavaScript";
  </script>
</body>
</html>
```

Cách viết mã lệnh JavaScript

- Mã lệnh được viết trong file có phần mở rộng là **js** và được nhúng vào file HTML thông qua thuộc tính **src** của thẻ **<script>** (External JavaScript)



```
<> test.html x [play] [outline] ... JS test.js x JS test.js > ...
<> test.html > html > body > p#greeting
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 |   <title>Test</title>
5 </head>
6 <body>
7 |   <p id="greeting"></p>
8 |   <script src="test.js" />
9 </body>
10 </html>
1   var element = document.getElementById("greeting");
2   element.innerHTML = "Welcome to JavaScript";
```

Cách viết mã lệnh JavaScript

- Viết trực tiếp trong các sự kiện của thẻ HTML (Inline JavaScript).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Test</title>
</head>
<body>
  <p id="greeting"></p>

  <button onclick="document.getElementById('greeting').innerHTML='Welcome'">
    Show Greeting
  </button>
</body>
</html>
```

Quy tắc viết lệnh trong Javascript

- Một câu lệnh có thể viết trên một hoặc nhiều dòng.
- Các câu lệnh phải được phân cách nhau bởi dấu chấm phẩy (kết thúc câu lệnh bởi dấu chấm phẩy)
- Khối lệnh (code block) được viết trong cặp dấu { }
- Chú thích (comment) được viết theo một trong 2 cách:
 - Chú thích một dòng: sử dụng //
 - Chú thích nhiều dòng: sử dụng /* */
- Tên trong JavaScript phân biệt chữ hoa và chữ thường.

Nhập, xuất dữ liệu

- Nhập dữ liệu:
 - Sử dụng lệnh `prompt()`
 - Sử dụng lệnh `confirm()`
 - **Thông qua các đối tượng nhập dữ liệu của HTML**
- Xuất dữ liệu:
 - Sử dụng lệnh `document.write()`
 - Sử dụng lệnh `window.alert()`
 - Sử dụng lệnh `console.log()`
 - **Thông qua các phần tử HTML để hiển thị dữ liệu.**

Khai báo biến và hằng

- Khai báo biến
 - Sử dụng **var**
 - Sử dụng **let**
- Khai báo hằng
 - Sử dụng **const**
- Biến có thể sử dụng mà không cần phải khai báo

```
var msg = "Hello", a, b = 10;  
  
let x = 100, y = 200;  
  
name = "John Smith";  
  
const PI = 3.14;
```

Phép toán trong JavaScript

Phép toán 2 ngôi	+, -, *, **, /, %
Phép toán 1 ngôi	++, --
Phép toán so sánh	==, ===, !=, !==, >, >=, <, <=
Phép toán rẽ nhánh	?
Phép toán Nullish Coalescing	??
Phép toán logic	&&, , !
Phép gán	=
	+=, -=, *=, /=, %=, **=

Kiểu dữ liệu cơ bản

- Trong JavaScript, một biến có thể thuộc kiểu dữ liệu bất kỳ, tùy thuộc vào giá trị mà biến đang lưu giữ.
- Các kiểu dữ liệu cơ bản:
 - boolean
 - number
 - string
 - undefined
- Toán tử **typeof**: được sử dụng để trả về kiểu dữ liệu của một biến hoặc một biểu thức

Các cấu trúc điều khiển

- **Rẽ nhánh:**
 - Lệnh `if`
 - Lệnh `switch`
- **Lặp:**
 - Lệnh `for`
 - Lệnh `while`
 - Lệnh `do ... while`

Hàm (function)

- Định nghĩa hàm:

```
function functionName(parameter1, parameter2, parameter3,...) {  
    //Các Lệnh trong thân hàm  
}
```

- Sử dụng lệnh **return** để trả về giá trị cho hàm

```
function addNumbers(a, b) {  
    return a + b;  
}
```

```
let sum = addNumbers(10, 50);
```

Hàm (function)

- Lưu ý khi sử dụng hàm:
 - Cho dù hàm có tham số hay không, thì sau tên hàm đều phải có cặp dấu ()
 - Không bắt buộc phải có `return` trong hàm
 - Khi gặp lệnh `return` trong thân hàm, việc thực thi các lệnh trong hàm kết thúc tại vị trí đó.
 - Khi gọi hàm, phải có cặp dấu () sau tên hàm. Trong trường hợp mà không có cặp dấu (), ghi ra mã code của hàm.

Hàm (function)

Biểu thức hàm (function expression)

- Hàm có thể được định nghĩa dưới dạng một biểu thức và lưu trữ trong một biến.

```
const f = function(a, b) {  
    return a - b;  
}  
console.log(f(50, 20));
```

- Những hàm được định nghĩa theo cách này thường được gọi là hàm ẩn danh (anonymous function).

Hàm (function)

Rest parameter

- Rest parameter được sử dụng để định nghĩa một số lượng không xác định các tham số trong một hàm.
- Rest parameter được khai báo với ba dấu chấm (...) ở phía trước tên tham số.
- Khi khai báo một tham số là rest parameter, tất cả các giá trị truyền cho tham số sẽ được xử lý thông qua tham số này dưới dạng mảng.

```
function sumArray(...args) {  
    let s = 0;  
    for (let item of args)  
        s += item;  
    return s;  
}  
  
var sum = sumArray(1, 2, 3, 4);  
console.log(sum);
```


Hàm (function)

Sử dụng đối tượng `arguments`

- Là đối tượng có sẵn trong hàm của JavaScript.
- Chứa tất cả các tham số truyền vào của hàm dưới dạng một mảng.

```
function sumAll() {  
    let sum = 0;  
    for (let i = 0; i < arguments.length; i++) {  
        sum += arguments[i];  
    }  
    return sum;  
}
```

```
console.log(sumAll(1, 2, 3));  
console.log(sumAll(1, 2, 3, 4, 5, 6));
```

- **Lưu ý:** Nên sử dụng `Rest Parameter` thay vì sử dụng đối tượng `arguments` cho mục đích tương tự.

Mảng (array)

- Mảng được sử dụng để lưu trữ một tập hợp các giá trị.
- Các phần tử trong mảng được viết trong cặp dấu ngoặc vuông, phân cách nhau bởi dấu phẩy.
- Thông qua chỉ số của phần tử để truy cập đến một phần tử trong mảng theo cú pháp:

`array[index]`

Trong đó, chỉ số của phần tử (index) được tính bắt đầu từ 0.

Mảng (array)

- Khai báo và tạo mảng

```
//Cách 1: khai báo và gán tập các phần tử cho mảng  
const students = ["Nga", "Vinh", "Hải"]
```

```
//Cách 2: khai báo và gán giá trị cho từng phần tử  
const classes = [];  
classes[0] = "Tin K46";  
classes[1] = "Tin K47";
```

```
//Cách 3: Sử dụng từ khóa new  
const depts = new Array("CNTT", "Toán", "Kiến trúc");
```

- Sử dụng thuộc tính **length** để lấy độ dài (số phần tử) của mảng
Ví dụ: `console.log(students.length);`

Mảng (array)

- Một số lệnh thường dùng trong mảng

pop()	Xóa phần tử cuối cùng trong mảng <code>students.pop();</code>
--------------	--

push()	Bổ sung phần tử vào cuối mảng <code>students.push("Phong")</code>
---------------	--

shift()	Xóa phần tử đầu tiên trong mảng <code>students.shift();</code>
----------------	---

unshift()	Chèn thêm phần tử vào đầu mảng <code>students.unshift("Châu")</code>
------------------	---

delete	Xóa giá trị một phần tử trong mảng (không xóa phần tử) <code>delete students[0];</code>
---------------	--

reverse()	Đảo ngược các phần tử trong mảng <code>students.reverse()</code>
------------------	---

Mảng (array)

- Một số lệnh thường dùng trong mảng

splice() Thay đổi nội dung của một mảng bằng cách thêm, xóa hoặc thay thế các phần tử vào mảng.
`array.splice(start, deleteCount[, item1, item2,...])`

Ví dụ:

```
const numbers = [1, 2, 3, 6];
```

```
//Chèn 2 phần tử có giá trị 4, 5 vào vị trí thứ 3 trong mảng  
numbers.splice(3, 0, 4, 5);
```

```
//Xóa 2 phần tử đầu tiên của mảng  
numbers.splice(0, 2);
```

```
//Thay 2 phần tử đầu tiên của mảng bởi phần tử có giá trị 10  
numbers.splice(0, 2, 10);
```

Mảng (array)

- Một số lệnh thường dùng trong mảng

indexOf()

Tìm vị trí của phần tử đầu tiên trong mảng

```
const numbers = [7, 5, 9, 20, 9, 4];  
var index = numbers.indexOf(9);    //index = 2
```

lastIndexOf()

Tìm vị trí của phần tử cuối cùng trong mảng

```
const numbers = [7, 5, 9, 20, 9, 4];  
var index = numbers.lastIndexOf(9); //index = 4
```

includes()

Kiểm tra xem có hay không một phần tử trong mảng

```
const numbers = [7, 5, 9, 20, 9, 4];  
exists = numbers.includes(20);    //exists = true  
exists = numbers.includes(200);   //exists = false
```

Mảng (array)

- Sắp xếp phần tử trong mảng:

```
array.sort([compareFunction])
```

Trong đó:

- **compareFunction**: (tham số tùy chọn) là một hàm so sánh nhằm định nghĩa cách sắp xếp. Nếu tham số này không được cung cấp thì sắp xếp mặc định theo thứ tự tăng dần dựa trên chuỗi Unicode.

Mảng (array)

- Sắp xếp theo thứ tự tăng dần

```
const numbers = [7, 23, 5, 9, 100, 20];  
numbers.sort(function(a, b){ return a - b });  
console.log(numbers);           //Kết quả: [5, 7, 9, 20, 23, 100]
```

- Sắp xếp theo thứ tự giảm dần

```
const numbers = [7, 23, 5, 9, 100, 20];  
numbers.sort(function(a, b){ return b - a });  
console.log(numbers);           //Kết quả: [ 100, 23, 20, 9, 7, 5 ]
```


Mảng (array)

- Phép duyệt qua các phần tử trong mảng

- Sử dụng vòng lặp **for**

```
for (let index = 0; index < numbers.length; index++) {  
    console.log(numbers[index]);  
}
```

- Sử dụng vòng lặp **for ... of ...**

```
for (let item of numbers) {  
    console.log(item);  
}
```

- Sử dụng hàm **forEach()**

```
numbers.forEach(function(value, index, array){  
    console.log("item at ", index, " = ", value);  
})
```

Kiểu đối tượng (object)

- Đối tượng (object) trong JavaScript là một tập hợp các cặp **key:value** được viết trong cặp dấu { } phân cách nhau bởi dấu phẩy.

Trong đó:

- key**: được gọi là thuộc tính (property)
- value**: giá trị của thuộc tính. Giá trị của thuộc tính có thể là một **giá trị**, một **đối tượng** khác hoặc cũng có thể là một **hàm** (thường gọi là **phương thức**).

```
var person = {  
  fullName: "Nguyễn Văn Hải",  
  age: 35,  
  address: {  
    houseNumber: "43/1",  
    street: "Nguyễn Huệ",  
    city: "Huế"  
  },  
  languages: ["Vietnames", "English", "French"],  
  sayIntroduction: function() {  
    return "Hello. My name is " + this.fullName;  
  }  
}
```

{JSON}
JavaScript Object Notation

Kiểu đối tượng (object)

- **Phương thức của đối tượng:** phương thức mô tả 1 hành vi của đối tượng, được cài đặt như là 1 hàm trong đối tượng.

```
var person = {  
  fullName: "Nguyễn Văn Hải",  
  age: 35,  
  address: {  
    houseNumber: "43/1",  
    street: "Nguyễn Huệ",  
    city: "Huế"  
  },  
  languages: ["Vietnames", "English", "French"],  
  sayIntroduction: function() {  
    return "Hello. My name is " + this.fullName;  
  }  
}
```

Phương thức của đối tượng

```
var greeting = person.sayIntroduction();  
console.log(greeting);
```

Lời gọi đến phương thức
của đối tượng

Kiểu đối tượng (object)

- Để truy cập vào thuộc tính (key) của đối tượng, có 2 cách viết:
 - Cách 1: `objectName.PropertyName`
 - Cách 2: `objectName["propertyName"]`

```
person.fullName = "Trần Văn Thanh";  
person["age"] = 47;
```

- Nếu việc truy cập vào thuộc tính của đối tượng được thực hiện ngay bên trong phần khai báo của đối tượng thì nên sử dụng từ khóa **this** thay vì sử dụng tên của đối tượng.

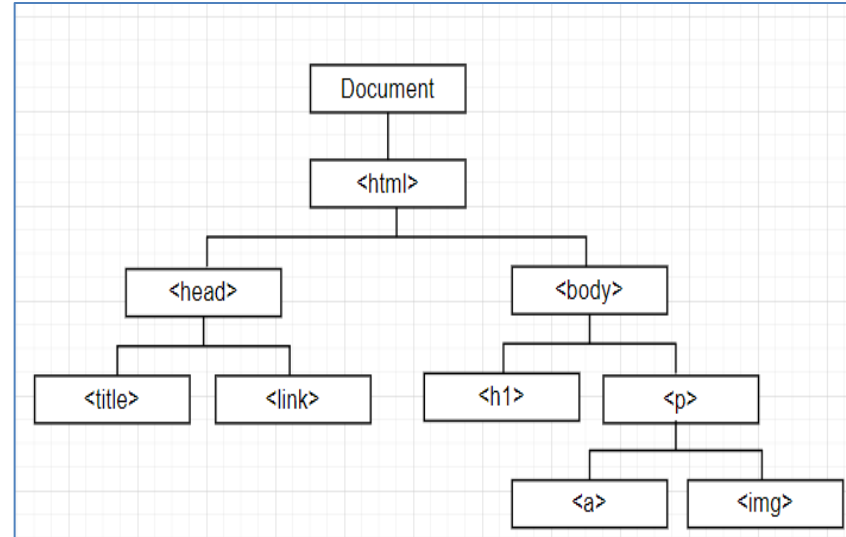
```
sayIntroduction: function() {  
    |   return "Hello. My name is " + this.fullName;  
    }  
}
```

Xử lý HTML DOM với JavaScript

HTML DOM là gì?

- DOM: Document Object Model (Mô hình đối tượng tài liệu)
- Khi hiển thị một trang web, trình duyệt sẽ tạo Mô hình đối tượng tài liệu (DOM) của trang dưới dạng cây

```
<html>
  <head>
    <title>My Title</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Welcome</h1>
    <p>
      <a href="#">Click here!</a>
      
    </p>
  </body>
</html>
```



HTML DOM là gì?

- HTML DOM là một mô hình và là giao diện lập trình chuẩn cho các tài liệu HTML.
- JavaScript có thể truy cập đến các phần tử trong tài liệu HTML và thực hiện các phép xử lý liên quan đến tài liệu HTML và các phần tử trong tài liệu.

Nguyên tắc chung khi xử lý HTML DOM

- Xác định (tìm) đúng phần tử cần xử lý bằng cách sử dụng các hàm:
 - `document.getElementById()`
 - `document.getElementsByTagName()`
 - `document.getElementsByClassName()`
 - `document.querySelector()`
 - `document.querySelectorAll()`
- Thực hiện các phép xử lý trên phần tử như:
 - Xử lý các thuộc tính của phần tử
 - Xử lý style, class CSS của phần tử
 - Bắt và xử lý các sự kiện (event) trên phần tử
 - ...

Xác định phần tử trong tài liệu HTML

- Xác định một phần tử bằng id

```
document.getElementById("id_của_thẻ")
```

Hàm trả về kết quả là **một phần tử**.

- Xác định các phần tử bằng tên thẻ

```
document.getElementsByTagName("tên_thẻ")
```

Hàm trả về kết quả là một **mảng các phần tử**.

- Xác định các phần tử bằng tên lớp

```
document.getElementsByClassName("tên_Lớp")
```

Hàm trả về kết quả là một **mảng các phần tử**.

Xác định phần tử trong tài liệu HTML

```
<html>
  <head>
    <title>Test</title>
  </head>
  <body>
    <p id="p-1" class="class-1">
      Trường Đại học Khoa học
    </p>
    <p id="p-2" class="class-2">
      Khoa Công nghệ Thông tin
    </p>
    <p id="p-3" class="class-2">
      Lớp Tin K44
    </p>
    <div id="div-1" class="class-1">
      Xin chào các bạn
    </div>
    <div id="div-2" class="class-2">
      Chào mừng các bạn đến với JavaScript
    </div>
  </body>
</html>
```

```
//p1: phần tử có id là p-1
var p1 = document.getElementById("p-1");
p1.style.color = "red";

//divTags: mảng các phần tử là thẻ div
var divTags = document.getElementsByTagName("div");
for (let tag of divTags) {
  tag.style.fontStyle = "Italic";
}

//class2Tag: mảng các phần tử thuộc lớp có tên là class-2
var class2Tags = document.getElementsByClassName("class-2");
for (let tag of class2Tags) {
  tag.style.color = "green";
}
```

Trường Đại học Khoa học

Khoa Công nghệ Thông tin

Lớp Tin K44

Xin chào các bạn

Chào mừng các bạn đến với JavaScript

Xác định phần tử trong tài liệu HTML

- Xác định phần tử bằng cách sử dụng CSS selector:

- Trường hợp 1: Xác định 1 phần tử

```
document.querySelector("css_selector");
```

Hàm trả về kết quả là một phần tử

- Trường hợp 2: Xác định nhiều phần tử

```
document.querySelectorAll("css_selector")
```

Hàm trả về kết quả là một mảng các phần tử.

Lưu ý: Hàm `querySelector()` và `querySelectorAll()` không hỗ trợ cho các trình duyệt phiên bản cũ.

Xác định phần tử trong tài liệu HTML

```
<html>
  <head>
    <title>Test</title>
  </head>
  <body>
    <p id="p-1" class="class-1">
      Trường Đại học Khoa học
    </p>
    <p id="p-2" class="class-2">
      Khoa Công nghệ Thông tin
    </p>
    <p id="p-3" class="class-1">
      Lớp Tin K44A
    </p>
    <div id="div-1" class="class-2">
      Chào mừng bạn đến với JavaScript
      <p>Bài thực hành 1</p>
      <p>Bài thực hành 2</p>
    </div>
  </body>
</html>
```

```
//element: phần tử có id là p-1
var element = document.querySelector("#p-1");
element.style.color = "red";

//elements: mảng các phần tử là thẻ p là con
//của phần tử có id là div-1
var elements = document.querySelectorAll("#div-1 > p");
for (let e of elements) {
  e.style.color = "green";
}
```

Trường Đại học Khoa học

Khoa Công nghệ Thông tin

Lớp Tin K44A

Chào mừng bạn đến với JavaScript

Bài thực hành 1

Bài thực hành 2

Xử lý nội dung của phần tử

- Nội dung của một phần tử thường được hiểu là toàn bộ phần văn bản và mã HTML nằm bên trong phần tử đó (nằm giữa cặp thẻ mở và thẻ đóng).
- Để đọc hoặc thay đổi nội dung của một phần tử, sử dụng thuộc tính `innerHTML` của phần tử đó theo cú pháp:

`element.innerHTML`

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <p id="demo">Hello the <b>World</b></p>
    <script>
      const element = document.getElementById("demo");
      console.log(element.innerHTML);
      element.innerHTML = "Welcome to JavaScript";
    </script>
  </body>
</html>
```

Xử lý nội dung của phần tử

- Ngoài thuộc tính `innerHTML`, có thể đọc hoặc thay đổi nội dung văn bản của phần tử thông qua hai thuộc tính `innerText` và `textContent`. Tuy nhiên, cần lưu ý đến sự khác biệt giữa chúng.
 - **`innerText`**: Nội dung văn bản của phần tử, không bao gồm nội dung của các phần tử con bị ẩn bằng CSS.
 - **`textContent`**: Nội dung văn bản của phần tử, bao gồm cả nội dung của các phần tử con bị ẩn bằng CSS.

```
<p id="demo">Hello the <b style="display: none;">World</b></p>
<script>
  const element = document.getElementById("demo");
  console.log(element.innerHTML);    //Hello the <b style="display: none;">World</b>
  console.log(element.innerText);    //Hello the
  console.log(element.textContent);  //Hello the World
</script>
```

Xử lý thuộc tính của phần tử

- JavaScript cho phép đọc và thay đổi giá trị thuộc tính của phần tử bằng cách truy cập đến thuộc tính bằng cú pháp:

`element.attributeName`

Trong đó:

- `element`: Là phần tử cần xử lý.
- `attributeName`: Là tên của thuộc tính cần xử lý (ví dụ: `href`, `src`,...).

```
<a id="demoLink">
|   <img id="demoImage">
| </a>
| <script>
|   document.getElementById("demoLink").href = "MyLink.html";
|   document.getElementById("demoImage").src = "MyImage.png";
| </script>
```



```
<a id="demoLink" href="MyLink.html">
|   
| </a>
```

Xử lý thuộc tính của phần tử

- JavaScript cung cấp một số phương thức sau để xử lý thuộc tính của phần tử:

- Đọc giá trị thuộc tính:

```
element.getAttribute("attributeName")
```

- Thay đổi giá trị thuộc tính:

```
element.setAttribute("attributeName", value)
```

- Xóa thuộc tính khỏi phần tử:

```
element.removeAttribute("attributeName")
```


Xử lý style của phần tử

- Thuộc tính **style** của một phần tử là một đối tượng chứa các thuộc tính CSS của phần tử đó. Thông qua thuộc tính **style**, có thể truy cập để đọc và thiết lập giá trị cho các thuộc tính CSS của một phần tử thông qua cú pháp:

`element.style.propertyName`

Trong đó *propertyName* là tên của thuộc tính CSS.

Lưu ý: với những thuộc tính CSS mà trong tên có dấu gạch ngang thì bỏ dấu gạch ngang (ví dụ: `font-size`, `background-color` viết thành `fontSize`, `backgroundColor`)

- Để xóa thuộc tính CSS (inline) của phần tử, sử dụng phương thức `removeProperty()` của đối tượng **style** theo cú pháp:

`element.style.removeProperty("property-name")`

Lưu ý: sử dụng đúng tên thuộc tính CSS cần xóa (ví dụ: `font-size`, `background-color`).

Xử lý style của phần tử

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <h1 id="head">GIỚI THIỆU</h1>

    <h2 class="title">Trường Đại học Khoa học</h2>
    <p>Địa chỉ: 77 Nguyễn Huệ, TP Huế</p>
    <p>Điện thoại: (0234) 3823290</p>

    <h2 class="title">Khoa Công nghệ thông tin</h2>
    <p>Địa chỉ: Tầng 2, dãy nhà G</p>
    <p>Điện thoại: (0234) 3826767</p>
  </body>
</html>
```

```
document.getElementById("head").style.color = "blue";

const titles = document.getElementsByClassName("title");
for (let element of titles) {
  element.style.color = "red";
  element.style.fontSize = "25px";
}

const paragraphs = document.getElementsByTagName("p");
for (let element of paragraphs) {
  element.style.fontFamily = "Verdana";
  element.style.color = "green";
}
```

Xử lý class CSS cho phần tử

- Thuộc tính `classList` của phần tử là một đối tượng cung cấp một số phương thức để thêm, xóa và kiểm tra các class CSS của một phần tử HTML. Đối tượng `classList` có các phương thức thông dụng sau:

- Thêm một hoặc nhiều class CSS cho phần tử:

```
element.classList.add("className1", "className2", ...)
```

- Xóa một hoặc nhiều class CSS ra khỏi phần tử:

```
element.classList.remove("className1", "className2", ...)
```

- Thêm class CSS vào phần tử nếu nó không tồn tại, hoặc loại bỏ class nếu nó đã tồn tại

```
element.classList.toggle("className")
```

- Kiểm tra xem phần tử có chứa class CSS không (hàm trả về giá trị kiểu Boolean)

```
element.classList.contains("className")
```

- Thay thế một class CSS bằng một class CSS khác

```
element.classList.replace("oldClassName", "newClassName")
```

Xử lý sự kiện (event) trên phần tử HTML

- Một sự kiện (event) là một tác động nào đó lên phần tử, ví dụ:
 - Người sử dụng nhấp chuột lên phần tử
 - Người sử dụng thay đổi dữ liệu trong một ô nhập dữ liệu
- Mỗi một sự kiện được định nghĩa và nhận biết dựa vào tên của sự kiện (ví dụ: `click`, `dblclick`, `change`, `focus`,...)
- Lập trình xử lý sự kiện: “Bắt” các sự kiện xảy ra trên phần tử và lập trình để xử lý khi sự kiện đó xảy ra (thường thông qua hàm xử lý sự kiện).
- Thông qua lập trình xử lý sự kiện để tạo nên sự tương tác giữa người sử dụng với các phần tử, giữa các phần tử với nhau.

Xử lý sự kiện (event) trên phần tử HTML

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <div>
      
    </div>
    <div>
      <button onclick="turnLightOn()">Turn On</button>
      <button onclick="turnLightOff()">Turn Off</button>
    </div>

    <script>
      function turnLightOn() {
        document.getElementById("light").src = "images/light_on.gif";
      }
      function turnLightOff() {
        document.getElementById("light").src = "images/light_off.gif";
      }
    </script>
  </body>
</html>
```



Turn On

Turn Off

Xử lý sự kiện (event) trên phần tử HTML

- Bắt sự kiện trực tiếp trong thẻ HTML thông qua thuộc tính sự kiện

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <p id="greeting"></p>
    <button id="btnShowGreeting" onclick="showGreeting()">
      Show Greeting
    </button>

    <script>
      function showGreeting() {
        document.getElementById("greeting").innerHTML = "Hi! My name is JavaScript";
      }
    </script>
  </body>
</html>
```

Bắt sự kiện **click** xảy ra trên button.
Hàm **showGreeting** được gọi là hàm xử lý sự kiện

Xử lý sự kiện (event) trên phần tử HTML

- Bắt sự kiện bằng JavaScript

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <p id="greeting"></p>
    <button id="btnShowGreeting">
      Show Greeting
    </button>

    <script>
      var button = document.getElementById("btnShowGreeting");
      button.onclick = showGreeting;

      function showGreeting() {
        document.getElementById("greeting").innerHTML = "Hi! My name is JavaScript";
      }
    </script>
  </body>
</html>
```

- Xác định phần tử cần “bắt” sự kiện
- Bắt sự kiện trên phần tử sử dụng cú pháp:
element.eventName = functionName;

Trong đó:

- *element*: phần tử cần bắt sự kiện
- *eventName*: tên sự kiện (vd: onclick, onchange,...)
- *functionName*: tên của hàm xử lý sự kiện, là hàm được gọi khi sự kiện xảy ra (lưu ý không sử dụng cặp dấu ngoặc sau tên hàm)

Xử lý sự kiện (event) trên phần tử HTML

- Bắt sự kiện bằng JavaScript (sử dụng *anonymous function*)

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <p id="greeting"></p>
    <button id="btnShowGreeting">
      Show Greeting
    </button>

    <script>
      var button = document.getElementById("btnShowGreeting");

      button.onclick = function() {
        document.getElementById("greeting").innerHTML = "Hi! My name is JavaScript";
      };
    </script>
  </body>
</html>
```

- Xác định phần tử cần “bắt” sự kiện
- Bắt sự kiện trên phần tử sử dụng cú pháp:
`element.eventName = function() {`

`};`

Trong đó:

- `element`: phần tử cần bắt sự kiện
- `eventName`: tên sự kiện (vd: onclick, onchange,...)

Xử lý sự kiện (event) trên phần tử HTML

- Bắt sự kiện bằng cách sử dụng hàm `addEventListener`, sử dụng cú pháp:

```
element.addEventListener(event, function [, useCapture])
```

Trong đó:

- *element*: là phần tử cần xử lý sự kiện
- *event*: Tham số đầu tiên của hàm là một chuỗi cho biết tên của sự kiện cần xử lý (ví dụ: “click”, “change”, “mousedown”,...).
- *Lưu ý: không sử dụng tiền tố “on” trước tên của sự kiện.*
- *function*: Là tên của hàm được gọi khi sự kiện xảy ra (có thể sử dụng anonymous function để xử lý sự kiện)
- *useCapture*: Là tham số tùy chọn, nhận giá trị kiểu boolean nhằm xác định cơ chế bắt và lan truyền sự kiện:
 - **false**: sử dụng cơ chế Bubbling, sự kiện lan truyền từ con ra cha (false là mặc định).
 - **true**: sử dụng cơ chế Capturing, sự kiện lan truyền từ cha vào con.

Xử lý sự kiện (event) trên phần tử HTML

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <p id="greeting"></p>
    <button id="btnShowGreeting">
      Show Greeting
    </button>

    <script>
      var button = document.getElementById("btnShowGreeting");
      button.addEventListener("click", showGreeting);

      function showGreeting() {
        document.getElementById("greeting").innerHTML = "Hi! My name is JavaScript";
      }
    </script>
  </body>
</html>
```

Xử lý sự kiện (event) trên phần tử HTML

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <p id="greeting"></p>
    <button id="btnShowGreeting">
      Show Greeting
    </button>

    <script>
      var button = document.getElementById("btnShowGreeting");

      button.addEventListener("click", function(){
        document.getElementById("greeting").innerHTML = "Hi! My name is JavaScript";
      });
    </script>
  </body>
</html>
```

Xử lý sự kiện (event) trên phần tử HTML

- Khi bắt sự kiện nào đó trên một phần tử, ta thường phải chỉ định hàm sẽ được gọi khi sự kiện xảy ra. Hàm này được gọi là **hàm xử lý sự kiện**.
- Hàm xử lý sự kiện có thể có hoặc không có tham số.
- Tham số kiểu đối tượng sự kiện (Event Object) của hàm xử lý sự kiện (thường đặt tên là **event**) được sử dụng để cung cấp thông tin liên quan đến sự kiện xảy ra như:
 - Loại sự kiện
 - Phần tử chịu tác động của sự kiện
 - Các thuộc tính của sự kiện (như phím được nhấn, tọa độ chuột, ký tự được nhập,...)

Xử lý sự kiện (event) trên phần tử HTML

```
<html>
  <head>
    <title>Demo</title>
  </head>
  <body>
    <input type="text" id="txt" onkeypress="handleEventFunction(event)">
    <select name="cmb" id="cmb">
      <option value="1">Option 1</option>
      <option value="2">Option 2</option>
    </select>
    <button id="btn">Click Me!</button>

    <script>
      document.getElementById("cmb").onchange = handleEventFunction;
      document.getElementById("btn").addEventListener("click", handleEventFunction);

      function handleEventFunction(event) {
        console.log(event);
      }
    </script>
  </body>
</html>
```

Hàm xử lý sự kiện với tham số **event**

Xử lý sự kiện (event) trên phần tử HTML

- **Cơ chế Capturing và Bubbling trong JavaScript**

- Trong JavaScript, sự kiện có thể được bắt và lan truyền qua các phần tử theo một trong hai cơ chế:
 - **Bubbling**: khi sự kiện được kích hoạt trên phần tử con, nó sẽ lan truyền dần lên các phần tử cha của nó (đây là cơ chế mặc định).
 - **Capturing**: khi sự kiện được kích hoạt trên phần tử cha, nó sẽ lan truyền dần đến các phần tử con của nó.
- Cơ chế bắt và lan truyền sự kiện được chỉ định thông qua tham số `useCapture` của hàm `addEventListener`

`addEventListener(event, function, useCapture)`

Trong đó tham số `useCapture` là một giá trị kiểu Boolean như sau:

- `false` (mặc định): sử dụng cơ chế Bubbling
- `true`: sử dụng cơ chế Capturing
- Để ngăn chặn việc lan truyền sự kiện, trong hàm xử lý sự kiện ta sử dụng phương thức `stopPropagation()` trên đối tượng sự kiện.

Xử lý sự kiện (event) trên phần tử HTML

- Cơ chế Capturing và Bubbling trong JavaScript

```
<html>
  <head>
    <title>Demo</title>
    <style>
      div {
        min-height: 20px;
        border: 1px solid black;
        padding: 20px;
        margin: 20px;
        background-color: yellow;
      }
    </style>
  </head>
  <body>
    <div id="div-1">DIV-1
      <div id="div-2">DIV-2
        <div id="div-3">DIV-3
          <div id="div-4">DIV-4</div>
        </div>
      </div>
    </div>
  </body>
</html>
```

Sử dụng cơ chế Bubbling

```
var divs = document.getElementsByTagName("div");
for (let element of divs) {
  element.addEventListener("click", handleEventFunction, false);
}

function handleEventFunction(event) {
  console.log(this.id);
  this.style.backgroundColor = "green";
  event.stopPropagation();
}
```

Ngăn chặn việc lan truyền sự kiện

Xử lý sự kiện (event) trên phần tử HTML

- Một số sự kiện thường gặp:

Tên sự kiện	Giải thích
click	Xảy ra khi nhấp chuột (trái) trên một phần tử
dblclick	Xảy ra khi nhấp đôi chuột trên một phần tử
contextmenu	Xảy ra khi nhấp chuột phải trên một phần tử
mousedown	Xảy ra khi nhấn nút chuột trên một phần tử
mouseup	Xảy ra khi nhả nút chuột trên một phần tử
mousemove	Xảy ra khi chuột được di chuyển trên một phần tử
mouseover	Xảy ra khi chuột được di chuyển vào một phần tử
mouseout	Xảy ra khi chuột được di chuyển ra khỏi một phần tử

Xử lý sự kiện (event) trên phần tử HTML

- Một số sự kiện thường gặp:

Tên sự kiện	Giải thích
keypress	Xảy ra khi nhấn một phím
keydown	Xảy ra khi một phím được đè xuống
keyup	Xảy ra khi một phím được nhả ra
change	Xảy ra khi giá trị (value) của một phần tử nhập dữ liệu như <code><input></code> , <code><select></code> ,... bị thay đổi
focus	Xảy ra khi con trỏ nhập liệu được đưa vào phần tử
blur	Xảy ra khi con trỏ nhập liệu rời khỏi phần tử
submit	Xảy ra khi một form được gửi đi
reset	Xảy ra khi nút reset trong form được nhấn

Xử lý sự kiện (event) trên phần tử HTML

- Một số sự kiện thường gặp:

Tên sự kiện	Giải thích
copy	Xảy ra khi sao chép (copy) nội dung của phần tử
cut	Xảy ra khi cắt (cut) nội dung của phần tử
paste	Xảy ra khi dán (paste) nội dung vào phần tử
load	Xảy ra khi một phần tử đã được tải hoàn toàn
unload	Xảy ra khi một phần tử đang bị thoát
resize	Xảy ra khi kích thước cửa sổ trình duyệt thay đổi
scroll	Xảy ra khi thanh cuộn được cuộn trong phần tử

Xử lý sự kiện (event) trên phần tử HTML

- Để xóa “hành động bắt sự kiện” trên một phần tử, sử dụng hàm `removeEventListener` theo cú pháp

`element.removeEventListener(event, function)`

Trong đó:

- *`element`*: là phần tử cần xử lý sự kiện
- *`event`*: Tham số đầu tiên của hàm là một chuỗi cho biết tên của sự kiện cần xử lý (ví dụ: “click”, “change”, “mousedown”,...).

Lưu ý: không sử dụng tiền tố “on” trước tên của sự kiện.

- *`function`*: Là tên của hàm đã được chỉ định để xử lý khi sự kiện xảy ra.